

Practical 2

State Management and Routing in React

Multi-Page SPA Navigation with react-router-dom, useState Hooks, and Controlled Form Inputs

Student Name:	Mantra Patel
Student ID:	24DCS076
Course:	Advanced Web Development Frameworks (ITUE301)
Institute:	DEPSTAR - CHARUSAT
Branch / Sem:	B.Tech CSE - Semester 5
Academic Year:	2025-2026
Dev Server Port:	http://localhost:5174

Objective: Implement client-side multi-page routing using react-router-dom and manage reactive UI state with useState in a student portfolio SPA.

1. Theory & Concepts

Practical 2 extends the portfolio with two core React capabilities:

* Client-side Routing (react-router-dom v7): Intercepts anchor clicks, updates the browser History API, and swaps the rendered component without triggering a server request or full page reload.

* useState Hook: Declares a reactive state variable and a setter function. Every setter call triggers React's virtual DOM reconciliation, updating only the affected DOM nodes.

- `<BrowserRouter>`: Wraps the app in main.jsx to provide routing context across the component tree.
- `<Routes>` / `<Route>`: Declarative route mapping - `path='/'` renders Home.jsx, `path='*'` renders NotFound.jsx (404).
- `<Link to='/projects'>`: Internal navigation anchor that prevents full page reload (replaces `<a href='...'>`).
- `useLocation()`: Hook returning the current URL path, used by NavBar to apply the .active CSS class.
- `useState(true)` - darkMode: Toggles between .app.dark and .app.light CSS classes on the root div.
- `useState(false)` - showTip: Controls the conditional rendering of the help tooltip in Contact.jsx.
- Controlled Inputs: `<input value={name} onChange={e => setName(e.target.value)} />` ensures React state is the single source of truth for form data.
- Live Character Counter: `{message.length} / 250` updates on every keystroke via controlled state.

2. Routing Architecture

Routing Hierarchy

```
main.jsx
  `-- <BrowserRouter>
      `-- App.jsx    (manages darkMode state)
          |-- NavBar.jsx  (Link components, active route detection)
          `-- <Routes>
              |-- Route path='/'          -> Home.jsx
              |-- Route path='/projects' -> Projects.jsx
              |-- Route path='/contact'   -> Contact.jsx
              `-- Route path='*'          -> NotFound.jsx (404)
```

3. Key Code Snippets

Dark/Light Mode Toggle - useState #1

```
// App.jsx - useState 1: Dark/Light theme toggle
const [darkMode, setDarkMode] = useState(true)
const toggleTheme = () => setDarkMode(prev => !prev)

return (
  <div className={darkMode ? 'app dark' : 'app light'}>
    <NavBar darkMode={darkMode} toggleTheme={toggleTheme} />
  </div>
)
```

```

    ...
  </div>
)

```

Controlled Inputs & Character Counter - useState #2+

```

// Contact.jsx - Controlled form inputs + useState
const [name, setName] = useState('')
const [email, setEmail] = useState('')
const [message, setMessage] = useState('')
const [showTip, setShowTip] = useState(false) // toggle #2

<input value={name} onChange={e => setName(e.target.value)} />
<textarea value={message} onChange={e => setMessage(e.target.value)} />
<p>{message.length} / 250 characters</p>    {/* live counter */}

```

NavBar.jsx - React Router Link & Active State

```

// NavBar.jsx - Link (not <a>) to prevent page reload
import { Link, useLocation } from 'react-router-dom'

const location = useLocation()

<Link to="/" className={location.pathname==='/' ? 'active' : ''}>
  Home
</Link>
<Link to="/contact" className={location.pathname==='/' ? 'active' : ''}>
  Contact
</Link>

```

4. Application Screenshots

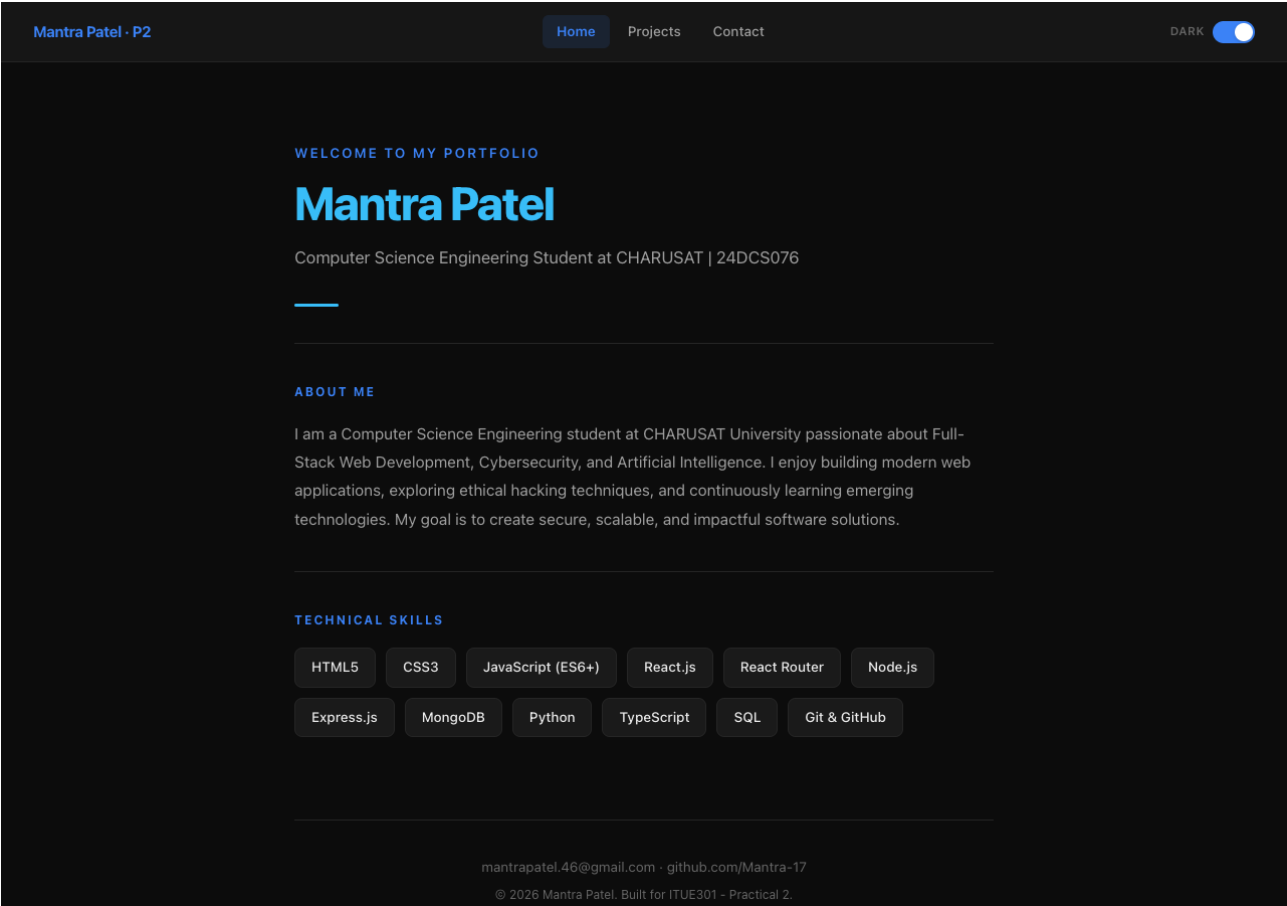


Figure: Practical 2 - Home Page (Dark Mode)

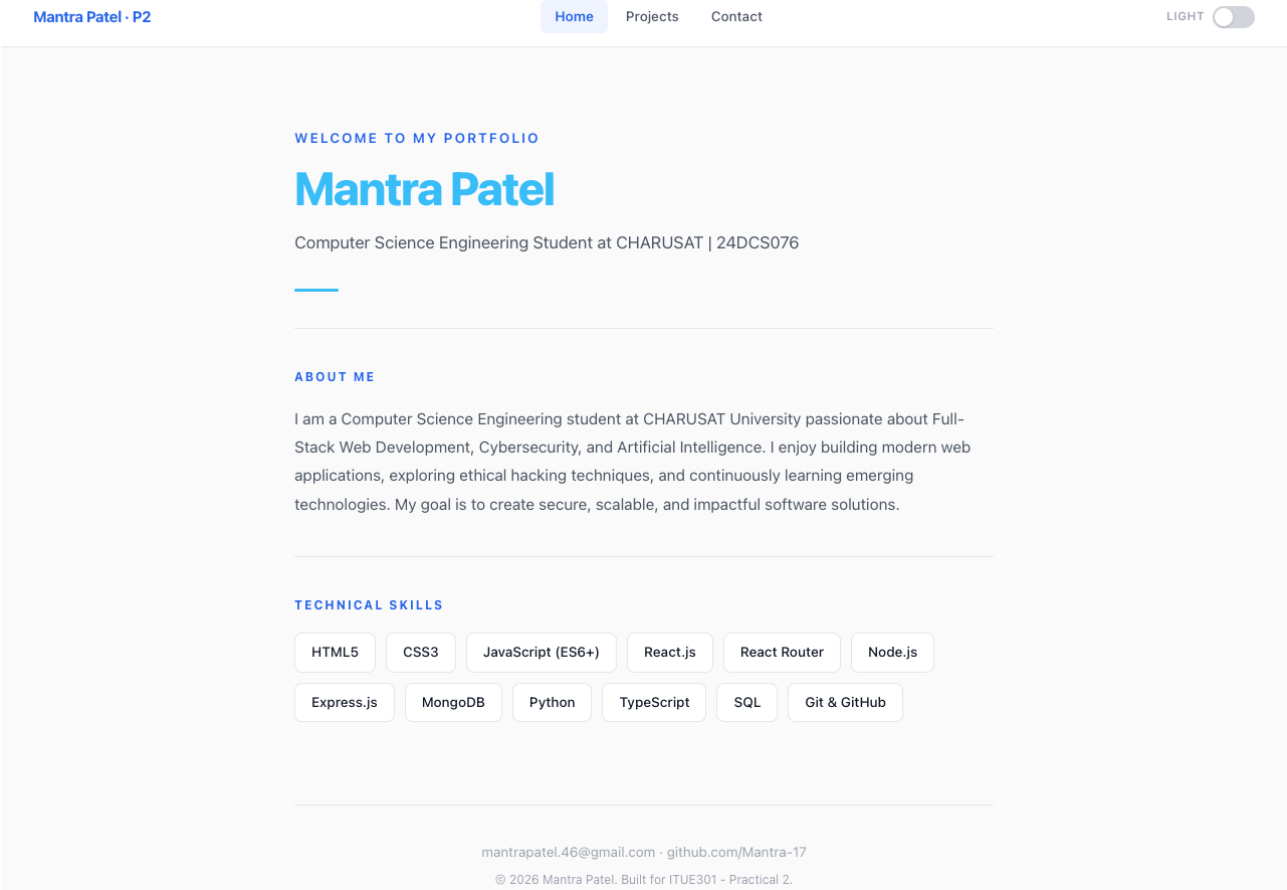


Figure: Practical 2 - Home Page (Light Mode after toggle)

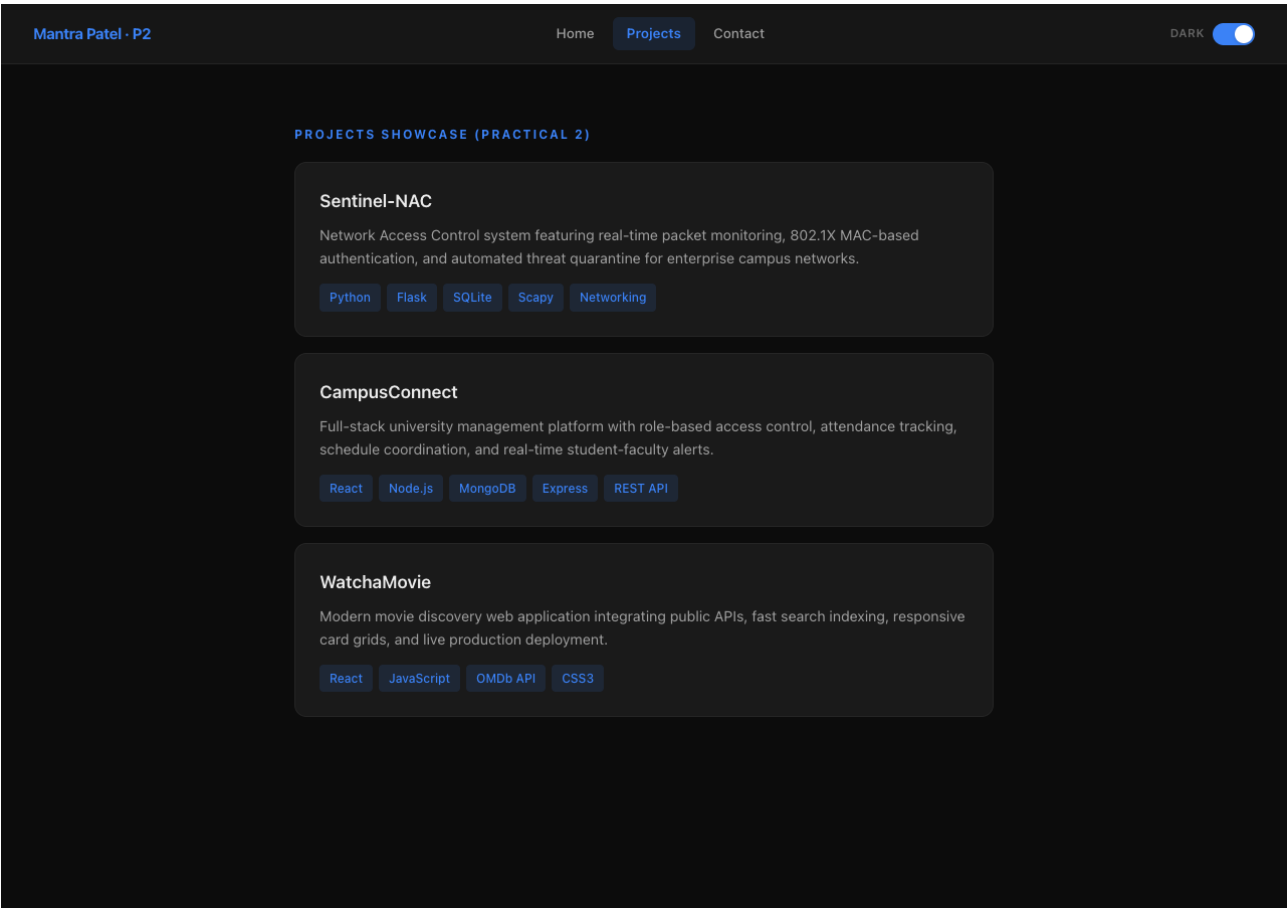


Figure: Practical 2 - Projects page (/projects route, no reload)

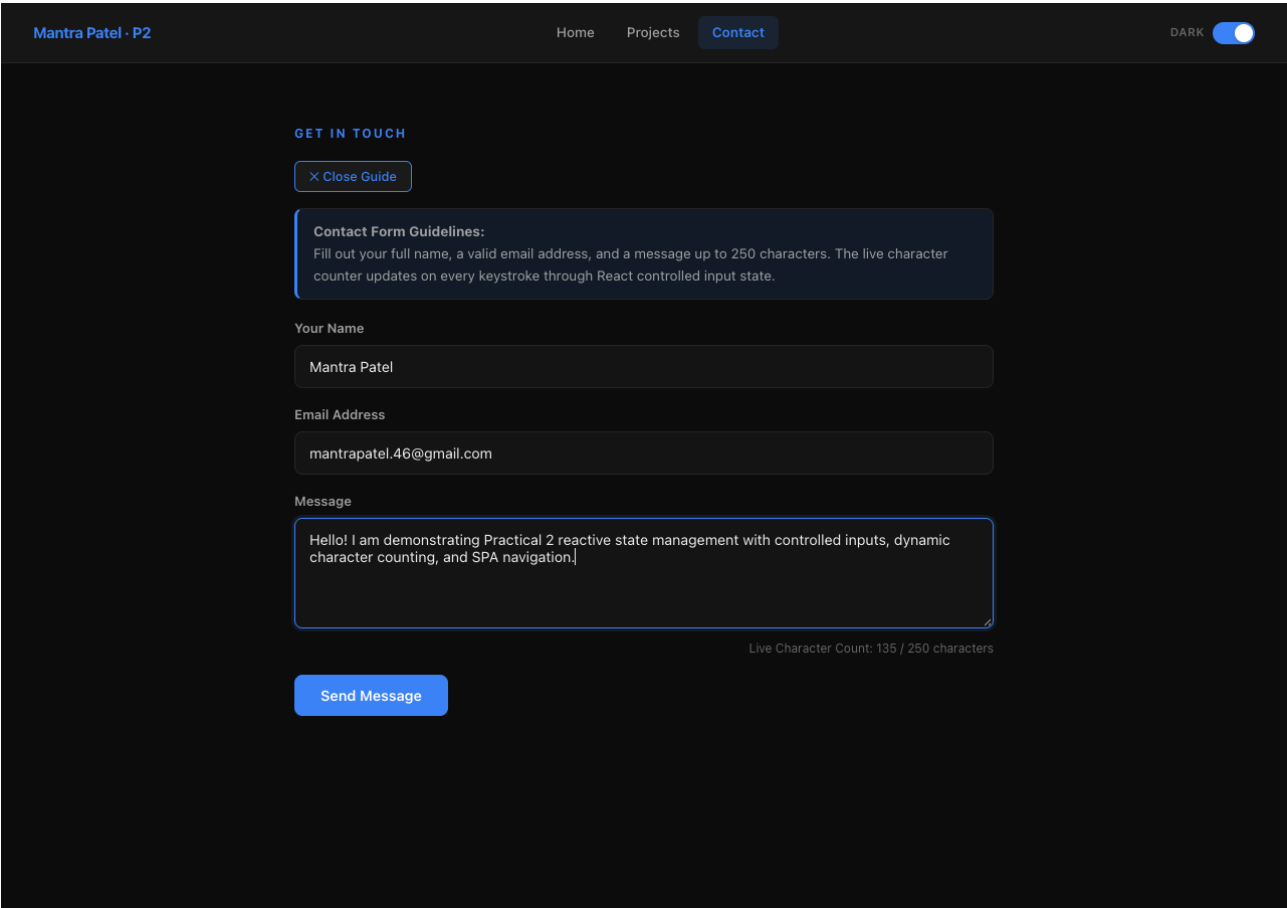


Figure: Practical 2 - Contact form with live character counter and help tooltip

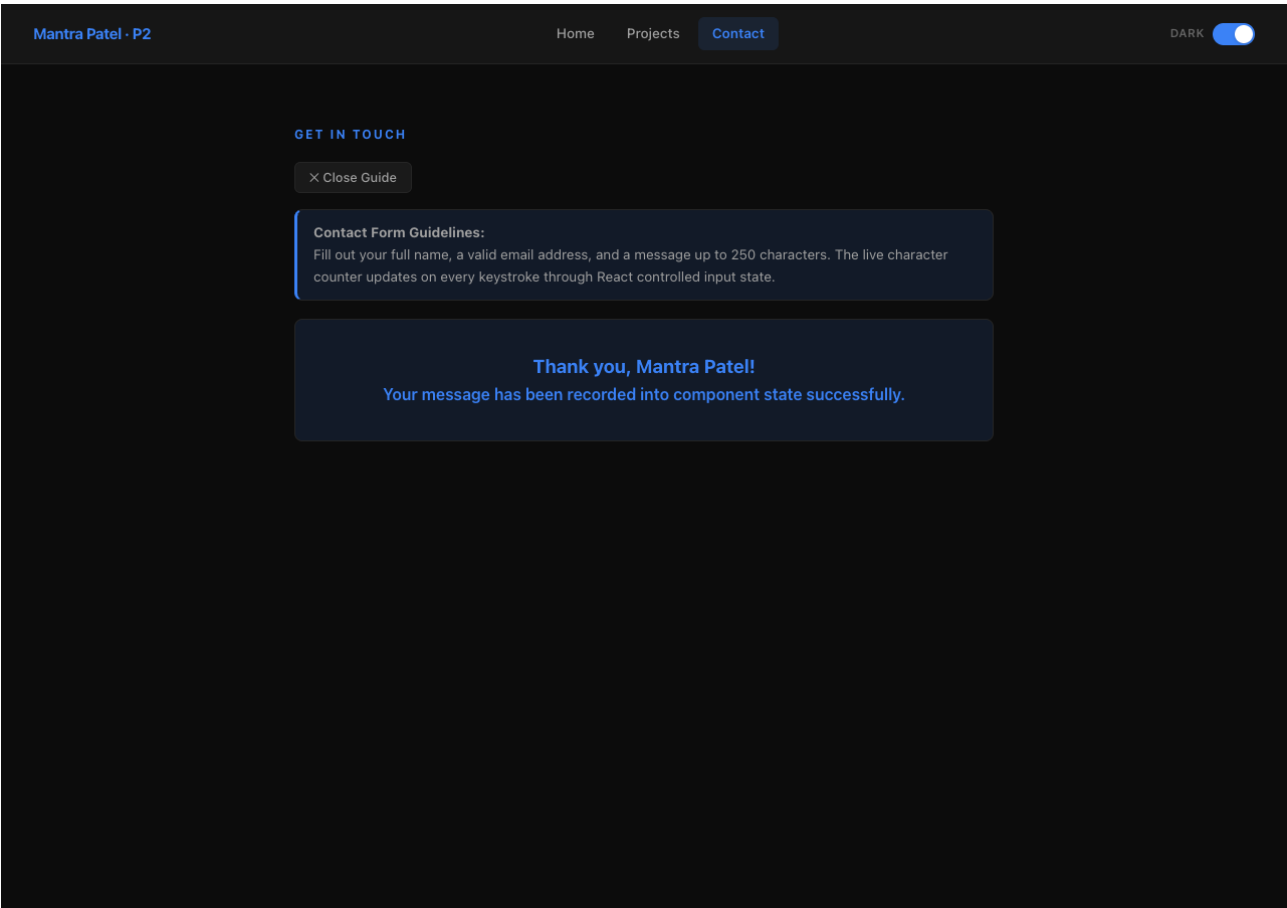


Figure: Practical 2 - Contact form submitted (state update confirmation)

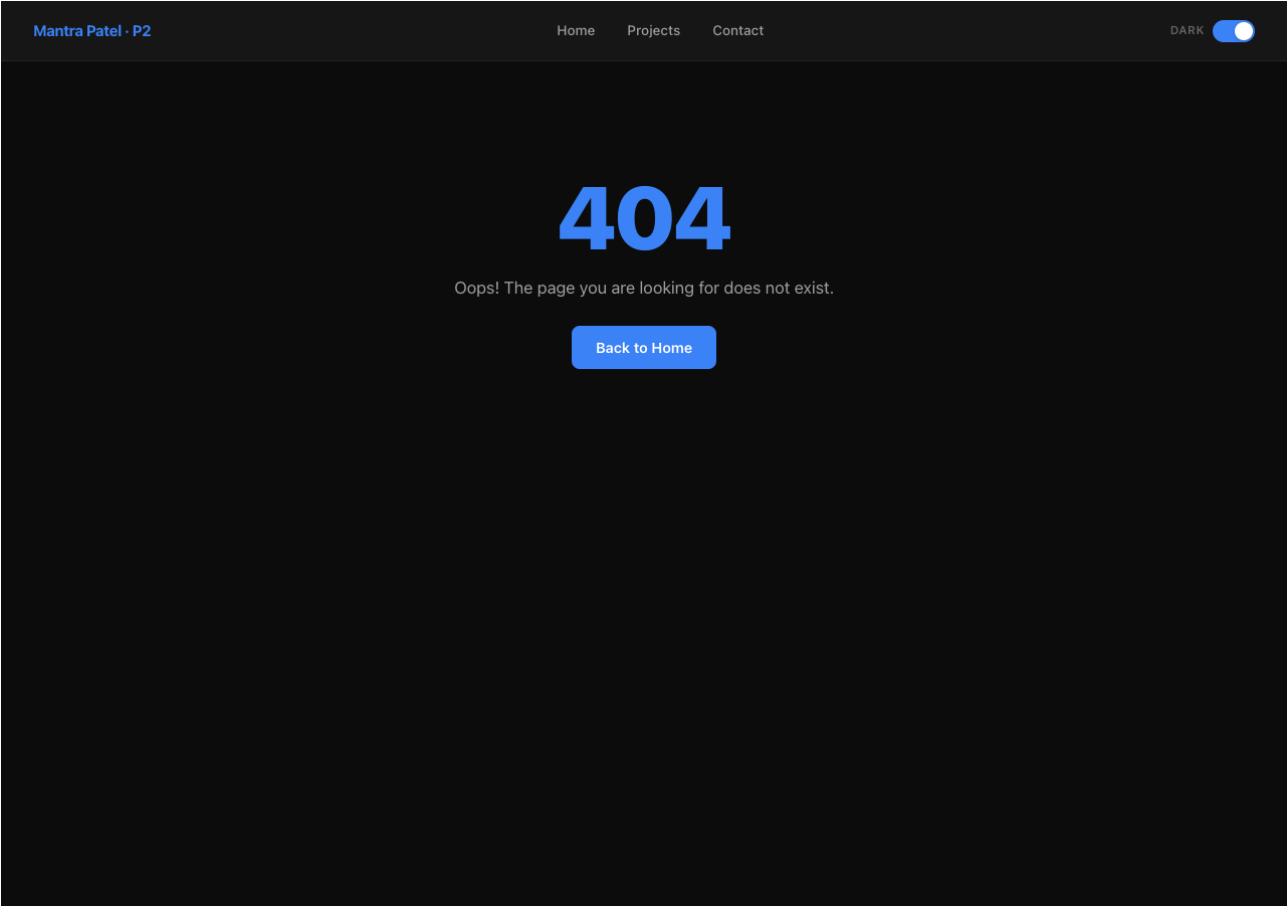


Figure: Practical 2 - 404 Not Found page (wildcard * route)

5. Testing Checklist

- npm install runs without errors in Practical-2/
- npm run dev launches on http://localhost:5174 with strictPort: true
- Navigating between /, /projects, and /contact has zero page reload
- Visiting /invalid-test-path renders the custom 404 NotFound page
- Dark/Light mode toggle smoothly transitions all colour tokens
- Contact form inputs are fully controlled: typing updates React state instantly
- Live character counter reflects exact message.length on every keystroke
- Help tooltip appears/hides correctly via showTip useState toggle
- Form submission shows confirmation message and resets all fields after 3 s
- Browser console: 0 errors and 0 React key warnings

6. Learning Outcomes

After completing Practical 2, I can:

- Implement client-side routing with react-router-dom using BrowserRouter, Routes, Route, and Link.
- Use the useLocation() hook to detect the active route and apply conditional CSS classes.
- Manage application-level state with useState and propagate it through props (dark mode).
- Build fully controlled form components where React state owns the input value at all times.
- Conditionally render JSX elements based on boolean state (help tooltip, confirmation message).
- Implement a catch-all 404 route using the * wildcard path in React Router.